

Mobile Application Development with Flutter

Lecture 59. Flutter Models & ViewModels

The Problem with Current API Code

```
var response = await http.get(...);  
  
var data = jsonDecode(response.body);  
  
print(data['name']);
```

- Problems:
 - Messy, hard to manage, repeated JSON keys, difficult to scale
- Solution:
 - Solution is to create models

What is a Model?

- A Model is a Dart class that represents data
- Suppose we've student data:

Name	Age	City
Ali	20	Lahore

- We create a class for this data
- That class is called a model

Simple Model Example

```
class StudentModel {  
    String name;  
    int age;  
    String city;  
  
    StudentModel({  
        required this.name,  
        required this.age,  
        required this.city,  
    });  
}
```

Creating Object

```
StudentModel student = StudentModel(  
    name: 'Ali',  
    age: 20,  
    city: 'Lahore',  
);
```

Accessing values:

```
print(student.name);
```

What is ViewModel?

- ViewModel handles:
 - Logic
 - APIs
 - Data preparation
 - State handling
- Important point
 - Model = Data
 - ViewModel = Logic

Why ViewModels Exist?

- Without ViewModel UI handles:
 - API calls
 - JSON parsing
 - Data storage
- Everything mixed together
- With ViewModel
 - UI becomes clean
 - ViewModel handles logic separately

Folder Structure

```
lib/  
|  
├─ models/  
|   └─ post_model.dart  
|  
├─ viewmodels/  
|   └─ post_viewmodel.dart  
|  
├─ screens/  
|   └─ home_screen.dart  
|  
└─ main.dart
```

Difference Between Model & ViewModel

Model	ViewModel
Stores data	Handles logic
Represents JSON	Handles APIs
Simple class	Business logic
No API logic	API communication
Example: UserModel	Example: UserViewModel